
**Diseño de un asistente virtual inclusivo para facilitar la
vida universitaria**

**Designing an inclusive virtual assistant to facilitate
university life**



Trabajo de Fin de Grado

Curso académico 2025–2026

Autores:

Sergio Serrano García

Adrián Redondo Gómez

Manuel Villa Abajo

Director:

Virginia Francisco Gilmartín

Codirector:

Antonio García Sevilla

Grado en Ingeniería Informática y Grado en Ingeniería del Software

Facultad de Informática - Universidad Complutense de Madrid

Nota obtenida (solo en la versión final):

Índice general

1	Introducción	3
1.1	Motivación	3
1.2	Objetivos	4
1.3	Plan de trabajo	5
1.4	Estructura de la memoria	5
2	Estado del arte	6
2.1	Discapacidad intelectual	6
2.1.1	Lectura fácil	7
2.2	Grandes Modelos de Lenguaje (LLMs)	10
2.2.1	Ingeniería de Contexto	10
2.2.2	Ingeniería de prompts	11
2.2.3	Retrieval-Augmented Generation (RAG)	15
2.2.4	Personalización del Modelo	16
3	Desarrollo (Provisional)	18
3.1	Autocompletado	18
3.2	Validación de resultados	19
4	Trabajo individual	21

1 Introducción

En este primer capítulo se presenta, en primer lugar, la motivación del proyecto, centrada en las dificultades que enfrentan los estudiantes con discapacidad intelectual en el entorno universitario. Después, se muestran los objetivos académicos, generales y específicos del trabajo, orientados al diseño y desarrollo de un sistema conversacional basado en inteligencia artificial con enfoque inclusivo. Para terminar, se describe el plan de trabajo seguido para su elaboración y la estructura general de la memoria.

1.1. Motivación

La universidad pública debe ser un espacio accesible e inclusivo para cualquier persona, independientemente de sus capacidades. Aun así, los estudiantes con discapacidad intelectual se suelen enfrentar a barreras significativas a la hora de desenvolverse en el entorno universitario: la complejidad de los procedimientos administrativos, la utilización de lenguaje demasiado técnico o la falta de recursos adaptados entre otros muchos problemas pueden dificultar la participación plena en la vida académica de estas personas.

La idea de este trabajo surge a partir de una problemática identificada de manera crítica por el equipo docente del programa ACCEDE¹ de la UCM, un título propio destinado para la formación de Técnicos Auxiliares en Evaluación de Entornos Inclusivos. Sus estudiantes han manifestado una dificultad persistente a la hora de comprender la estructura, los procesos administrativos y la operativa diaria tanto de la universidad en general como de la Facultad de Educación (donde cursan sus estudios) en particular.

Este Trabajo Fin de Grado abordará esta brecha de inclusión mediante el desarrollo de un Sistema Conversacional Inteligente (Chatbot) que use las capacidades de los Modelos de Lenguaje Avanzados para servir tanto como un recurso de apoyo para el alumnado del título ACCEDE, como para ofrecer un diálogo guiado y proactivo que responda las preguntas con claridad y desglose los procesos paso a paso. Así, los alumnos del título ACCEDE podrán desenvolverse con mayor autonomía dentro del entorno universitario, comprendiendo mejor los trámites, servicios y dinámicas institucionales.

¹Página web de ACCEDE: <https://www.ucm.es/programa-accede/>

1.2. Objetivos

El objetivo principal de este trabajo es la creación de un sistema conversacional accesible, orientado a dar apoyo a estudiantes con discapacidad intelectual en la resolución de dudas sobre el funcionamiento de la Universidad Complutense de Madrid y sus Facultades en general, y de la Facultad de Educación en particular. Este sistema permitirá facilitar el acceso a información sobre cómo solicitar tutorías, cómo acceder al campus virtual o cómo sacar un libro de la biblioteca a todas las personas con discapacidad intelectual. Aunque el público objetivo de nuestro sistema es el colectivo de estudiantes con discapacidad intelectual, creemos que nuestro asistente beneficiará la comprensión de la información y la autonomía de todos los estudiantes de la universidad. Para conseguir este objetivo general, nos planteamos los siguientes objetivos específicos:

- Desarrollar una base de conocimiento específica a partir de fuentes oficiales, como web institucionales, manuales o guías de uso.
- Explorar y seleccionar las estrategias de integración más efectivas entre la base de conocimiento y el LLM, incluyendo técnicas de recuperación de información (RAG), prompt engineering y/o enfoques híbridos.
- Adaptar las respuestas generadas por el sistema a lectura fácil.
- Integrar recursos visuales de apoyo, como pictogramas procedentes de repositorios especializados (por ejemplo, ARASAAC), para reforzar la comprensión de la información.
- Evaluar la calidad y efectividad del sistema.
- Involucrar a los estudiantes del programa ACCEDE en el proceso de validación y mejora del sistema, recogiendo sus opiniones para adaptar el chatbot a sus necesidades reales.

Además, el proyecto contempla los siguientes objetivos académicos:

- Aprender a integrar LLMs con arquitecturas RAG, técnicas de ingeniería de prompts y métodos de personalización como Fine-Tuning y LoRA para conseguir desplegar un ecosistema de Inteligencia Artificial.
- Adquirir competencias avanzadas en el desarrollo de software y el diseño de interfaces conversacionales.
- Reflexionar sobre la dimensión ética del desarrollo de la inteligencia artificial, asegurando que la tecnología no sea solo usable, sino que fomente la autonomía y la inclusión en el entorno universitario.
- Profundizar en la naturaleza de la discapacidad intelectual, comprendiendo cómo las barreras del entorno limitan la participación social y fomentar la autonomía y la igualdad de oportunidades en el ámbito universitario de todos los estudiantes.

1.3. Plan de trabajo

1.4. Estructura de la memoria

La presente memoria se organiza de la siguiente manera:

- **Capítulo 1: Introducción.** Se expone la motivación, los objetivos y la estructura general de la memoria.
- **Capítulo 2: Estado del arte.** Se revisan los principales avances en modelos de lenguaje grandes, y se estudia la discapacidad intelectual en relación con el uso de la inteligencia artificial, además de analizar la biblioteca de la Facultad de Educación de la UCM.
- **Capítulo 3: Desarrollo.** Se detallan las etapas de la creación del asistente conversacional.
- **Capítulo 4: Resultados.** Se presentan los resultados cualitativos y cuantitativos obtenidos por el sistema.
- **Capítulo 5: Conclusiones y líneas futuras.** Se resumen los logros obtenidos en el trabajo y se plantean posibles mejoras y líneas de investigación futuras.
- **Capítulo 6: Contribuciones personales.** Se describe el trabajo realizado por cada miembro del equipo y las responsabilidades asumidas durante dicha realización.

2 Estado del arte

En el presente capítulo se establece la base teórica y tecnológica sobre la que se sustenta la propuesta del asistente virtual accesible. Para ello, este capítulo aborda dos ramas necesarias para establecer los fundamentos teóricos y técnicos que han guiado la implementación de la solución propuesta. En primer lugar, se profundiza en el concepto de la discapacidad intelectual y en las necesidades específicas y barreras habituales en el acceso a la información que experimenta este colectivo, mencionando temas como la Lectura Fácil y la accesibilidad cognitiva, y después, se habla de los LLMs (Grandes Modelos de Lenguaje), que constituyen la base tecnológica del asistente.

2.1. Discapacidad intelectual

La discapacidad no tiene únicamente una visión sanitaria, en la actualidad también tiene una visión multidimensional incluyendo aspectos sociales, legislativos y tecnológicos. Según la Clasificación Internacional del Funcionamiento, de la Discapacidad y de la Salud (CIF) y la Clasificación Internacional de Enfermedades en su versión más reciente, CIE-11 (OMS, 2023), la discapacidad se entiende como el resultado de la interacción entre tres elementos:

- Deficiencia de una estructura o función corporal.
- Limitación de las actividades cotidianas.
- Restricción de la participación en la sociedad.

Legalmente se considera persona con discapacidad aquella que presenta deficiencias físicas, mentales, intelectuales o sensoriales, previsiblemente permanentes que al interactuar con barreras, puedan impedir su participación plena y efectiva en la sociedad. Además, para considerarse legalmente una persona con discapacidad tiene que tener un grado de discapacidad igual o superior a 33 %. La discapacidad intelectual (DI) es uno de los tipos concretos de deficiencias previsiblemente permanentes.

Las personas con discapacidad intelectual presentan déficits en diversas funciones mentales generales como en el razonamiento y resolución de problemas, planificación y anticipación de consecuencias, pensamiento abstracto, juicio y toma de decisiones, así como en el aprendizaje tanto académico como a partir de la experiencia. Estas limitaciones afectan a su capacidad para comprender información compleja y adquirir nuevos conocimientos.

Además del funcionamiento intelectual, la discapacidad intelectual también implica limitaciones en el funcionamiento adaptativo, es decir, en la capacidad para desenvolverse de manera autónoma en la vida diaria. Estas dificultades se dan en tres ámbitos diferentes:

- Habilidades conceptuales, como la comprensión del lenguaje, el manejo de la información o la organización de las tareas.
- Habilidades sociales, relacionadas con la comunicación, la interpretación de normas sociales y la participación en la comunidad.
- Habilidades prácticas, que incluyen el autocuidado, la gestión del hogar y el desempeño en actividades académicas o laborales.

De acuerdo con el modelo de la Clasificación Internacional del Funcionamiento (CIF), la discapacidad no depende únicamente de las características individuales, sino también de la interacción con el entorno. Existen diferentes barreras que limitan la inclusión plena de las personas con DI, como entornos poco comprensibles, escasa accesibilidad cognitiva, dificultades comunicativas (uso de lenguaje excesivamente complejo o falta de apoyos visuales) o falta de comprensión social hacia la diversidad cognitiva.

Las personas con discapacidad intelectual tienen una serie de dificultades generales que afectan concretamente al uso de herramientas tecnológicas. Entre ellas se encuentran las dificultades para navegar por las interfaces, ya que la mayoría requieren la habilidad de orientarse en menús con estructuras complejas, recordar rutas de navegación, identificar jerarquías de información o comprender la relación entre acciones y resultados. Asimismo, se observan dificultades en la toma de decisiones, derivadas de la sobrecarga cognitiva presente en numerosas interfaces debido al exceso de opciones simultáneas, múltiples estímulos visuales y la necesidad de mantener información en memoria a corto plazo. Por último, existen dificultades en la comprensión de la información, dado que muchas herramientas utilizan un lenguaje complejo, textos abstractos, instrucciones ambiguas o carecen de apoyos visuales.

2.1.1. Lectura fácil

La lectura fácil es un método de redacción que tiene como objetivo facilitar el acceso a la información, la cultura y la literatura, lo cual está garantizado como un derecho fundamental, a cualquier persona, independientemente de su edad, capacidad intelectual o nivel de idioma. Busca eliminar las barreras que impiden la comprensión, el aprendizaje y la participación [1]. Al hacer la información accesible, la lectura fácil ayuda a las personas a tomar sus propias decisiones y a ejercer otros derechos, como el de participación ciudadana. Aunque la lectura fácil es útil para el conjunto de la sociedad, hay colectivos a los que les es especialmente útil. Entre ellos se encuentran:

- Personas con discapacidad intelectual
- Personas con enfermedades y trastornos mentales y del comportamiento.

- Personas mayores que pueden llegar a tener deterioro cognitivo
- Población inmigrante que tengan un dominio limitado del idioma
- Personas con analfabetismo o bajo nivel de alfabetización
- Personas con dislexia.
- Personas con dificultades del desarrollo del lenguaje por discapacidad auditiva
- Niños hasta cuarto curso de educación básica.

La metodología de la Lectura Fácil está respaldada por la Norma UNE 153101:2018 EX [2], la cual establece los requisitos para la elaboración de documentos en este formato. La adhesión a esta norma garantiza la calidad y la estandarización del proceso de adaptación. A continuación presentamos un resumen de las pautas recogidas en la norma de la UNE:

- **Nivel de Lenguaje y Vocabulario**
 - **Simplicidad Léxica:** Se priorizará el uso de un vocabulario sencillo y de uso cotidiano, evitando términos poco frecuentes o palabras en otros idiomas, a menos que sean muy conocidas.
 - **Claridad Conceptual:** Se evitará el uso de metáforas (ej. "llueve a cántaros"), frases hechas, abstracciones o jerga innecesaria que no sea literal.
 - **Coherencia Terminológica:** Se mantendrá una consistencia estricta, utilizando siempre la misma palabra para referirse al mismo concepto en todo el escrito.
 - **Gestión de Abreviaturas y Siglas:** Se evitarán iniciales y abreviaturas (como "etc." o "p.e."). Si es imprescindible usar una sigla, debe explicarse completamente la primera vez.
 - **Uso de Pronombres:** Se debe tener especial cuidado con los pronombres (él, eso, ella); si no queda totalmente claro a qué se refieren, se debe repetir el nombre propio o el sujeto.
- **Estructura Sintáctica y Estilo**
 - **Estructura Oracional Simple:** Las frases serán cortas, expresando una única idea por frase y separándolas con un punto en lugar de comas o conjunciones.
 - **Voz Activa y Directa:** Se utilizará la voz activa y un lenguaje directo, dirigiéndose al lector con "tú" o "usted".
 - **Formas Verbales:** Se recomienda el uso de verbos en tiempo presente y modo indicativo, evitando el pasado y el uso del subjuntivo.
 - **Redacción Positiva:** Se priorizarán las frases afirmativas sobre las negativas, ya que son más fáciles de procesar cognitivamente.

- **Gestión de Números y Fechas:** Los números se escribirán siempre con cifras (no con letras o números romanos). Las fechas deben aparecer completas (ej. martes 13 de octubre de 2012).
- **Diseño Visual y Tipografía**
 - **Tipografía Accesible:** Se usarán fuentes claras y sin remates (**sans-serif**), como Arial o Tahoma. Se prohíbe el uso de cursivas y mayúsculas sostenidas.
 - **Tamaño y Énfasis:** El tamaño mínimo de la letra será de 14 puntos. Se usará la **negrita** para destacar términos clave, evitando el subrayado excepto para enlaces.
 - **Contraste y Fondo:** El texto debe tener un fuerte contraste con el fondo. No se deben utilizar fotos o estampados detrás del texto que dificulten la lectura.
- **Maquetación y Distribución del Espacio**
 - **Alineación:** El texto se alineará siempre a la **izquierda** sin justificar, manteniendo espacios regulares entre palabras para evitar distracciones.
 - **Corte de Frases:** No se deben separar palabras al final de una línea. Si una frase ocupa dos líneas, el corte debe realizarse donde se haría una pausa natural al hablar.
 - **Organización de Páginas:** Se evitarán las columnas y las notas al pie de página. Las páginas deben numerarse indicando el total (ej. “página 2 de 4”).
 - **Jerarquía de Información:** La información más importante debe aparecer al principio del documento o destacada en cuadros de texto.
- **Elementos de Apoyo Visual**
 - **Imágenes Coherentes:** Se usarán imágenes, dibujos o símbolos que apoyen directamente el texto, situándolas siempre próximas al párrafo que explican.
 - **Adecuación al Público:** Las imágenes deben ser de buena calidad y no ser infantiles si el documento está dirigido a personas adultas.
 - **Identificación:** Los documentos deben incluir el logotipo europeo de lectura fácil para que los usuarios puedan identificar el formato.
- **Proceso de Validación**
 - **Participación de Usuarios:** Un documento solo se considera de Lectura Fácil si personas con discapacidad intelectual han participado en su revisión y han validado que la información es comprensible.

La mejor forma de validar si un texto tiene lectura fácil es incorporar al proceso de revisión a personas con dificultades de comprensión lectora para saber si el texto es fácil de leer.

2.2. Grandes Modelos de Lenguaje (LLMs)

Los grandes modelos de lenguaje (LLM, por sus siglas en inglés) son sistemas de inteligencia artificial capaces de leer y generar texto de forma similar a una persona, a partir de enormes cantidades de ejemplos de lenguaje humano. Estos modelos aprenden patrones del idioma (cómo se forman las frases, se relacionan las ideas o qué palabras suelen aparecer juntas) y, gracias a ello, pueden responder preguntas, resumir textos o explicar conceptos sin necesidad de programarlos desde cero para cada tarea. Su comportamiento es ajustable mediante instrucciones en lenguaje natural y parámetros de inferencia como la temperatura (que regula la creatividad), el top-p (que limita las palabras por probabilidad acumulada) o el top-k (que restringe a las más probables), lo que permite adaptar la variabilidad y precisión del texto generado. Para representar el significado del lenguaje de forma numérica, estos modelos emplean embeddings, vectores de alta dimensión que codifican la información contextual de los datos de forma que elementos con significados similares se sitúan próximos en el espacio vectorial. Gracias a esta versatilidad, los LLM se han convertido en una herramienta clave para el desarrollo de asistentes virtuales y aplicaciones que facilitan el acceso a la información de forma personalizada y accesible.

Este apartado se centrará en aquellos aspectos de los LLM más relevantes para el diseño del asistente virtual, abordando la ingeniería de prompts, la generación aumentada por recuperación (RAG), la ingeniería de contexto y las estrategias de personalización del modelo mediante fine-tuning.

2.2.1. Ingeniería de Contexto

En los LLM existe lo que se conoce como context window, define la cantidad máxima de información que el modelo puede tener en cuenta simultáneamente al generar una respuesta y que se utiliza como la memoria a corto plazo del modelo. El context engineering se basa en darle al LLM el contexto necesario, lo más relevante posible evitando información irrelevante, optimizando el uso del espacio disponible, para que el modelo pueda dar una respuesta precisa. Existen diferentes tipos de contexto que hay que tener en cuenta a la hora de crear una aplicación con un LLM[3]:

- Instrucciones, como prompts del sistema que definen el rol y el comportamiento del modelo, el texto introducido por el usuario y la memoria a corto plazo de la conversación para mantener una continuidad en el diálogo.
- Conocimiento, actúa como una memoria a largo plazo para información relevante como historiales de conversaciones anteriores, documentos externos o información de una base de conocimientos.
- Herramientas (tools), es la información generada por herramientas externas invocadas por el LLM como consultas a bases de datos o llamadas a APIs, la cual se añade al contexto.

A la hora de intentar implementar esto en una aplicación con un LLM hay que tener especial cuidado con que el contexto sea relevante y que este tenga espacio en la context window de nuestro modelo. Para conseguir eso hay unas cuantas técnicas que se pueden utilizar[4]:

- **Knowledge base or tool selection:** En el context engineering se usan múltiples bases de conocimiento. Antes de recuperar contexto de estas el modelo recibe información sobre qué se encuentra en cada una y así nuestro modelo elige correctamente.
- **Context ordering or compression:** Una limitación importante es el espacio que tenemos en la context window dado que no toda la información recuperada puede añadirse directamente al contexto, esta técnica reduce el volumen de información mediante resumirla. En algunas implementaciones no solo importa lo que aparece sino también el orden, ya que la posición de la información puede influir en la atención del modelo. Por esto se añade un paso intermedio en el que los fragmentos se ordenan por relevancia antes de añadirlo al contexto, priorizando la información más significativa.
- **Structured Information:** Para evitar dar contexto innecesario y reducir el uso de la context window, se puede utilizar structured outputs para conseguir únicamente el contexto relevante. Para esto, se define un esquema que el LLM debe seguir en su salida de modo que la información se represente de forma estructurada y pueda utilizarse posteriormente como contexto adicional de forma más eficiente. Por ejemplo, si un LLM da información sobre películas, en lugar de añadir descripciones largas de cada película, se le puede pedir que genere un output estructurado siguiendo un esquema como título, director, año. De esta forma, solo se incluye en el contexto la información relevante,

2.2.2. Ingeniería de prompts

La ingeniería del prompt, o ingeniería de peticiones, se ha consolidado como un área clave en el desarrollo y personalización de sistemas basados en modelos de lenguaje avanzados. Esta disciplina, en constante evolución, se centra en el diseño estratégico de instrucciones, contextos y ejemplos para optimizar la interacción y el rendimiento de la inteligencia artificial en tareas específicas. Para conseguir una salida acorde a las necesidades del usuario, los prompts que se deben crear deben ser específicos y precisos.

La clave para que la calidad de las respuestas sea la mejor posible es realizar un prompt con una estructura que incluya elementos como los siguientes [5]:

- **Rol:** Se le asigna una identidad al modelo para establecer el tono y el estilo de la respuesta.
- **Instrucciones:** Descripción clara de la tarea a realizar o de las directrices que debe seguir el modelo.

- **Contexto:** Información relevante que ayuda al modelo a entender la situación y generar una respuesta más precisa.
- **Datos de entrada:** Información sobre la que se debe trabajar.
- **Indicador de resultado:** Indica cómo se debe estructurar la respuesta.

Ejemplo de Prompt Estructurado

Rol: Actúa como un bibliotecario

Instrucciones Responde siempre en Lectura Fácil con frases cortas y usando pasos numerados para procedimientos.

Contexto [Estamos en la biblioteca de la Universidad y tu deber es responder a las preguntas que los alumnos puedan tener sobre el funcionamiento de la biblioteca]

Datos de entrada: ¿Cómo pido una tutoría con un profesor?

Indicador de resultado: Procedimiento completo en pasos sencillos.

Para conseguir aprovechar al máximo el potencial de los modelos de lenguaje y reducir errores, se recomiendan pautas como las siguientes:

- Emplear instrucciones claras y concisas.
- Proporcionar contexto relevante.
- Indicar el formato o enfoque deseado para la respuesta.
- Empezar la petición con una pregunta o directiva.
- Incluir ejemplos de respuesta cuando sea oportuno.
- Dividir tareas complejas en subtareas manejables.
- Experimentar con diferentes estructuras y ser creativo.
- En algunos casos es recomendable realizar lo que se conoce como prompt negativo, en donde se explica lo que no se quiere incluir en la respuesta, y así el modelo evita comportamientos no deseados o la aparición de información indeseada.

Además de la correcta formulación de las peticiones (prompts) y el ajuste de sus parámetros de inferencia, la optimización del rendimiento de los modelos de lenguaje se consigue mediante el uso de diversas técnicas de prompting estratégicas. Estas técnicas definen el nivel de orientación y contextualización que se proporciona al modelo, lo cual es crucial para guiar su razonamiento y la calidad de la respuesta. Entre las metodologías más reconocidas para mejorar la interacción con los LLMs destacan:

- **Zero Shot Prompting (ZSP):** Esta técnica explota la capacidad intrínseca del LLM para ejecutar una tarea a partir de una instrucción clara y concisa, sin la

necesidad de proporcionar ejemplos previos o entrenamiento específico sobre dicha tarea. El modelo genera la respuesta basándose exclusivamente en el amplio conocimiento lingüístico y contextual adquirido durante su preentrenamiento. La efectividad del ZSP reside en su capacidad para la generalización y la interpretación contextual, confiando en que la instrucción es suficiente para evocar el comportamiento deseado.

Ejemplo de Zero Shot Prompting

Datos de entrada: ¿Cómo saco un libro de la biblioteca?

- **Few Shot Prompting:** El objetivo de esta técnica es guiar al modelo ya entrenado previamente proporcionándole una pequeña cantidad de ejemplos completos (normalmente entre dos y cinco) que muestren claramente la forma de resolver una tarea específica. Estos ejemplos sirven como instrucciones contextuales, mostrando explícitamente el formato, el estilo y el tipo de razonamiento esperado.

Ejemplo de Few-Shot Prompting (FSL)

Pregunta: ¿Cómo saco un libro de la biblioteca?

Respuesta: Elige el libro que quieras sacar y ve al mostrador para que los bibliotecarios lo registren y puedas sacarlo.

Pregunta: ¿Cuándo puedo sacar un libro de la biblioteca?

Respuesta: Puedes sacar un libro de la biblioteca casi todos los días.

- Lunes a viernes: Abre a las **9:00** y cierra a las **20:30**.
- Sábados: Abre de **9:00** a **14:00**.

- **One Shot Prompting (OSL):** A menudo considerado un caso específico de Few-Shot Prompting, se define por la inclusión de un único par completo de ejemplo-respuesta dentro de la petición enviada al modelo. Esta técnica se emplea cuando la tarea a realizar requiere un formato de salida muy específico o un estilo conversacional particular. Al proporcionar una demostración precisa y bien estructurada, el modelo aprende rápidamente el formato, la sintaxis y el tono deseado, reduciendo la ambigüedad y la necesidad de instrucciones puramente textuales complejas. Si bien no es tan robusto como el Few-Shot con múltiples ejemplos, el OSL es una estrategia altamente eficiente en términos de ventanas de contexto y recursos, ofreciendo un equilibrio óptimo entre orientación del modelo y economía de datos.

Ejemplo de One-Shot Prompting (OSL)

Pregunta: ¿Cuándo puedo sacar un libro de la biblioteca?

Respuesta: Puedes sacar un libro de la biblioteca casi todos los días.

- Lunes a viernes: Abre a las **9:00** y cierra a las **20:30**.
- Sábados: Abre de **9:00** a **14:00**.

- **Chain-of-Thought (CoT):** Esta técnica tiene como objetivo desbloquear capacidades avanzadas de razonamiento en los modelos de lenguaje, instruyéndoles para que generen una serie de pasos lógicos intermedios antes de emitir la respuesta final. El CoT es especialmente eficaz para abordar tareas complejas que requieren múltiples etapas de procesamiento, como problemas matemáticos o la desestructuración y adaptación de textos. Esta metodología asegura que el modelo no solo acierte la respuesta, sino que también muestre un proceso de análisis transparente y estructurado.

Ejemplo de Chain-of-Thought (CoT)

Pregunta: ¿Cómo saco un libro de la biblioteca?

Indicador de Resultados: Responde a la pregunta mostrando el razonamiento paso a paso, mostrando cómo piensas la secuencia de acciones para lograr el objetivo.

Respuesta:

- Entra a la biblioteca y coge el libro que quieras.
- Escoge el libro de las estanterías y llévalo al mostrador.
- Entrega el carnet y registrarán el préstamo.
- Te dirán cuándo devolverlo y podrás llevartelo hasta ese día.

- **Role Prompting:** consiste en instruir al LLM para que adopte una identidad específica (un rol) con el objetivo de dirigir su estilo de respuesta, su tono y el enfoque de su conocimiento. Esta técnica es fundamental para la coherencia y la calidad de los asistentes virtuales especializados, ya que delimita el contexto de actuación del modelo, haciendo que sus salidas sean más relevantes y consistentes.

Ejemplo de Prompting por Asignación de Rol (Role Prompting)

Rol: Actúa como el bibliotecario de la biblioteca de la universidad, cuya misión es ayudar a estudiantes con cualquier duda que puedan tener.

Datos de entrada: ¿Cómo saco un libro de la biblioteca?

- **Styled/Theme-Controlled Prompting:** Técnica utilizada para imponer un conjunto de restricciones formales y estilísticas específicas a la salida del LLM. Su objetivo es garantizar que la respuesta generada se adhiera a un formato pre-definido, un tono particular, o un estándar de redacción.

Ejemplo de Prompting por Control de Estilo (Themed-Controlled)

Estilo Usa frases muy cortas, vocabulario cotidiano, y divide los procedimientos en pasos numerados o viñetas. Evita tecnicismos y alinea el texto a la izquierda.

Datos de entrada: ¿Cómo saco un libro de la biblioteca?

2.2.3. Retrieval-Augmented Generation (RAG)

Los LLM tienen diferentes problemáticas, como que su información está fijada en el tiempo, lo que puede dar lugar a que sea poco reciente o específica. Además, pueden generar respuestas incorrectas cuando la información no se encuentra en su base de conocimiento o respuestas inexactas por confusiones terminológicas al usar en diferentes entrenamientos terminologías similares para referirse a elementos diferentes[6].

Para mejorar estos problemas se utiliza la técnica de generación aumentada por recuperación (RAG) consiste en extender las bases de conocimiento originales de un LLM para conseguir ampliarla a un dominio específico sin necesidad de reentrenar el modelo completo[7].

En un sistema RAG se emplea un vector store, una librería o servicio encargado de almacenar embeddings y de realizar búsquedas por proximidad en el espacio vectorial mediante algoritmos de vecino más cercano aproximado (Approximate Nearest Neighbor)[8]. Esta búsqueda permite localizar los vectores similares reduciendo el coste computacional frente a una búsqueda exacta a cambio de una mínima pérdida en la precisión de esta.

Una vez la base de conocimiento está indexada como vectores, el RAG opera en dos fases principales. En la fase de recuperación (retrieval) se calcula el embedding del prompt realizado y se busca en el vector store los vectores más próximos. En la fase de generación (generation), los fragmentos recuperados se incorporan al contexto del modelo generativo. Con esa información adicional el modelo puede producir respuestas más precisas

y mejor fundadas. Aún así, esto no garantiza la veracidad de la respuesta obtenida, el modelo puede seguir generando contenido que no pertenezca al vector store si los datos que se recuperan son inconsistentes o insuficientes.

Para la base de conocimiento que se utiliza, es necesario aplicar una técnica llamada chunking. Esta técnica permite dividir un documento muy largo en partes más pequeñas, denominadas chunks. Esto permite realizar recuperaciones más precisas, ya que los fragmentos pequeños son más fáciles de relacionar con el prompt, aunque puede aumentar el número total de elementos almacenados. Hay una gran variedad de técnicas para hacer chunking, algunas de las más relevantes son[9]:

- **Sliding Window Chunking:** Crea chunks que se superponen; por ejemplo, en el primer chunk estarían las frases 1 y 2, y en el segundo, la 2 y la 3.
- **Semantic Chunking:** Utiliza embeddings o machine learning para dividir el texto por tema o idea para que cada chunk sea coherente con su significado. Por ejemplo, en un documento sobre un sistema de control industrial un primer chunk puede contendría la definición general del sistema, otro la descripción de los sistemas y un tercero el algoritmo de control empleado.
- **Subdocument Chunking:** Resume el documento entero o grandes partes, posteriormente estas partes se dividen en chunks añadiendo la información de la sección entera como metadatos de cada chunk.

Aunque los vector store recuperen los documentos más cercanos en el espacio de embeddings esto no garantiza que estos fragmentos sean los más útiles para el prompt. Para rebajar la probabilidad de que esto pase se utiliza el reranking, una técnica que reorganiza los documentos recuperados para priorizar aquellos que realmente son más relevantes para el prompt. Para implementar este proceso, el RAG incorpora un paso adicional en el que otro modelo vuelve a evaluar los candidatos recuperados y los ordena según su verdadera relevancia. Con esto se reduce la pérdida de información, se mejora la detección de matices lingüísticos y se incrementa la calidad de los fragmentos que finalmente se pasan al modelo generativo[10].

2.2.4. Personalización del Modelo

A pesar de la amplia capacidad de los LLM para generar texto de forma similar a una persona, a menudo es necesario ir más allá de las técnicas de prompting y RAG para adaptar su comportamiento a requisitos muy específicos.

- **Fine-Tuning (FT):** Esta técnica busca adaptar el comportamiento y el estilo de respuesta de un modelo que ha sido previamente entrenado con un corpus masivo de datos genéricos. El objetivo del FT es que el modelo realice tareas o actividades más específicas y/o adapte un tono conversacional concreto, logrando así una personalización del comportamiento del modelo base, permitiendo ahorrar tiempo y recursos significativos en comparación con entrenar un LLM desde cero.

El FT funciona de la siguiente manera. Primero debemos escoger un modelo de lenguaje robusto y preentrenado que ya sepa realizar tareas de lenguaje de manera general. Después debemos presentarle el conjunto de datos con los que queremos entrenarlo, siendo éste más pequeño y específico, que contiene ejemplos de cómo queremos que se comporte el modelo, y durante el FT, se ajustan los parámetros de las capas del modelo preentrenado, permitiendo así recalibrar el modelo para que las nuevas salidas se parezcan más a los ejemplos de nuestro conjunto de datos especializado.

- **Técnicas de Ajuste Fino Eficientes (PEFT):** Las Técnicas de Ajuste Fino Eficientes (PEFT) son métodos esenciales para mejorar el rendimiento de los LLMs y las redes neuronales ya entrenadas, adaptándolas a tareas o conjuntos de datos específicos. Funciona congelando la mayoría de los parámetros y capas del modelo preentrenado, mientras se añaden y capacitan un pequeño conjunto de parámetros entrenables, conocidos como **adaptadores**, en las capas finales. Su importancia radica en que equilibra la eficiencia y el rendimiento, ya que al capacitar solo un subconjunto reducido de parámetros, PEFT logra un ahorro significativo de tiempo y recursos computacionales, en comparación con el FT tradicional [11]. Esto permite crear modelos altamente especializados a una fracción del costo y en menos tiempo, y dando como resultado LLMs accesibles, flexibles y especializados.
- **Low-Rank Adaptation (LoRA):** Una de las técnicas más conocidas de PEFT es LoRA. Cuando los modelos de inteligencia artificial tienen miles de millones de parámetros, ajustar esos parámetros no es nada fácil, y puede ser costoso, tanto en tiempo como en recursos. De aquí nace la técnica LoRA (Low-Rank Adaptation), técnica que permite adaptar rápidamente un modelo base sin necesidad de modificar todos sus parámetros originales. LoRA, al ser un PEFT, primero “congela” el modelo preentrenado y además, añade matrices de bajo rango llamadas matrices de descomposición gemelas junto a las capas originales [12]. Estas matrices son las que se entrenan, reduciendo aun más la cantidad de parámetros entrenables.

El utilizar LORA facilita la personalización de grandes modelos para nuevas tareas, hace posible el FT en equipos con recursos limitados y en tiempos más reducidos, y se trata de una solución eficiente y accesible que mantiene el rendimiento del modelo a la perfección.

3 Desarrollo (Provisional)

*TANTO LA ORGANIZACIÓN DEL PUNTO EN SÍ COMO EL CONTENIDO ES PROVISIONAL.

Este capítulo presenta tanto las decisiones tomadas como el uso de las tecnologías pertinentes, para la creación del sistema conversacional que ayudará a los estudiantes con discapacidad a integrarse con mayor facilidad en la vida universitaria.

3.1. Autocompletado

Para que la función de autocompletar sea eficiente, rápida e intuitiva, en el front-end deberá de salir como una continuación de la palabra o grupos de palabras en un color un tono grisáceo (asumiendo texto principal en negro), y si el usuario pulsa el tabulador en el teclado, el texto sugerido pasará a formar parte del texto que el usuario está redactando. El autocompletado funcionará hará su funcionalidad más importante desde el Backend y llamando a un modelo de inteligencia artificial de la siguiente manera:

1. El usuario va escribiendo en la página lo que quiere preguntar.
2. El frontend envía el texto parcial al backend.
3. El backend llama al LLM.
4. El modelo genera una continuación con el contexto de la frase que ya haya escrito.
5. El backend devuelve la sugerencia al frontend.
6. El frontend la muestra como autocompletado, para que al pulsar el tabulador se escriba solo.

La inteligencia artificial que se utilice para realizar el autocompletado debe de ser distinto al que nos de las respuestas del usuario, ya que el LLM que se encarga de dar las respuestas requiere de un contexto largo, razonamiento y coherencia, además de que tiene que dar respuestas largas y complejas basándose en los prompts predefinidos para crear la respuesta y modificarla para que se muestre en Lectura Fácil. El autocompletado, por el contrario, requiere de baja latencia y de completado corto, por lo que simplemente tiene que dar sugerencias cortas y directas. Por ello, hay que buscar un LLM que de respuestas rápidas: Microsoft Phi-3 o Phi-4 (Mini), DistilGPT-2 o Hugging Face "SmolLM" parecen buenos para esta tarea.

Buenas prácticas:

1. **Añadir debounce al input:**

Evitamos que el usuario tenga que solicitar el proceso de recibir el autocompletado pulsando alguna tecla. Cada 300-500ms en los que el usuario no esté escribiendo, se manda una solicitud al LLM.

2. **Buscar un modelo SaaS gratuito, o que tenga la uni**

3. **Limitar los tokens:**

Hay que poner los tokens bajos, porque no hace falta que devuelva párrafos. Un máximo de tokens.

4. **Hay que hacer que el backend filtre saltos de línea o símbolos raros, para que no los incluya como sugerencia.**

5. **Usar tanto temperatura como top-p bajos**

La temperature debería de ser literalmente 0.

6. **No enviar contexto excesivo**

7. **Se puede optimizar utilizando caché por contexto con Redis**

Si ya se usó una sugerencia para el mismo contexto, no hace falta volver a hacer una petición. Usar Redis estaría bastante cool.

3.2. Validación de resultados

La validación de modelos de inteligencia artificial ayuda a evaluar el rendimiento de un modelo entrenado para garantizar que el objetivo para el que se creó y para el que se utiliza se cumple. En este proyecto, es necesario realizar más de un tipo de validaciones para comprobar que las salidas que ofrezca el modelo sean las adecuadas.

Tipos de validaciones:

- **Validación factual:** El modelo debe de responder correctamente a preguntas sobre horarios, préstamos y renovaciones, reservas de salas, trámites... La validación factual es el hecho de evaluar si un texto generado como el nuestro se ajusta estrictamente a la realidad de los datos, garantizando que la información sea veraz, actualizada y libre de alucinaciones[13].

Para ello, la validación debe de ser manual (método vanilla), basándonos en reglamentos internos, FAQs, la página web, y meter esta documentación en el RAG y que lo haga él mismo. Se podría crear un conjunto de entradas y comparar la

respuesta esperada con la respuesta real, y también se puede comparar por desviaciones críticas.

Si no lo hacemos con el RAG, podríamos hacer que gente que trabaje en la propia biblioteca, (expertos), validan la veracidad, actualidad y pertinencia de los datos. Los mejores que pueden saber si los datos que está devolviendo el modelo son buenos y tienen sentido son los propios bibliotecarios que trabajan allí.

- **Validación de Accesibilidad Cognitiva:** Hay que buscar la manera en la que poder hacer una evaluación de legibilidad automática. Con esto se podrá medir la longitud media de las frases, la complejidad léxica e incluso la presencia de tecnicismos que no se expliquen correctamente. INFLESZ, Análisis de vocabulario poco frecuente y conteo de subordinadas son algunos métodos a tener en cuenta. Otra manera es validar las respuestas con especialistas en accesibilidad cognitiva, pero parece más complicado de hacer y de gestionar. También se puede validar con usuarios reales, es decir, los estudiantes del grupo ACCEDE. Para ello se pueden hacer preguntas de verificación tras leer una respuesta, poner métricas (tasas de acierto, tiempo, aclaraciones solicitadas).
- **Validación conversacional:** Hay que validar también, que la conversación y las respuestas que el modelo ofrece sean útiles y coherentes en el diálogo, no solo correctas. Hay que ver que el modelo sea coherente y relevante. Además hay que evaluar si el asistente sabe cuando "no sabe" si redirige bien a un bibliotecario humano, a un correo electrónico o a una sección oficial.
- **Validación de seguridad y límites:** Hay que evitar cosas como las siguientes:
 - Hay que rechazar información inventada, servicios inexistentes y, en definitiva, respuestas fuera del dominio.
 - Evitar violencia, discriminación, acoso y contenido inapropiado.
 - Hay que diseñar casos para detectar respuestas inventadas y que no alucine, ya que la respuesta debe de ser segura, no inventada y con alternativas reales.

4 Trabajo individual

****PROVISIONAL, se irá añadiendo información y redactando mejor ****

- Sergio Serrano:
Punto 2.1 - Discapacidad intelectual, punto 2.2.3 - Retrieval-Augmented Generation (RAG), punto 2.2.1 - Ingeniería de Contexto. Trastear con Ollama.
- Adrián Redondo:
Punto 2 - Párrafo introductorio, punto 2.1.1 - Lectura Fácil. Investigación sobre las técnicas de prompting. Trastear con Ollama.
- Manuel Villa:
Portada, Estructura de la memoria, Punto 1 , 2.2 - Párrafo introductorio, 2.2.2 - Prompt Engineering y 2.2.4 - Personalización del Modelo Investigación sobre las técnicas de prompting. Investigación sobre el FT, el PEFT y LoRA. Trastear con FT, con Prompts y con Ollama. Investigación sobre el autocompletado y sobre la validación de los resultados del modelo. Hacer que con LOrA, técnicas PEFT, y FSL prompting se obtengan resultados en Lectura Fácil.

Bibliografía

- [1] Plena Inclusión. ¿qué es la lectura fácil?, 4 2024.
- [2] Plena Inclusión. Información para todos, 3 2021.
- [3] LangChain Accounts. Context engineering, 7 2025.
- [4] Tuana Çelik and Logan Markewich. Context engineering - what it is, and techniques to consider, 7 2025.
- [5] Dair-Ai. Github - dair-ai/prompt-engineering-guide: Guides, papers, lessons, notebooks and resources for prompt engineering, context engineering, rag, and ai agents.
- [6] AWS. ¿qué es rag?: explicación de la ia de generación aumentada por recuperación.
- [7] Sharma, Sandeep. Understanding rag: Evolution, components, implementation, and applications — by sandeep sharma — medium, 1 2024.
- [8] Yashwanth S. Retrieval augmented generation (rag) – 04: Vector stores and vector databases — by yashwanth s — medium, 5 2025.
- [9] Mastering LLM (Large Language Model). 11 chunking strategies for rag - simplified visualized — by mastering llm (large language model) — medium, 11 2024.
- [10] MyScale. Mejorando los modelos rag con reranking y langchain, 5 2024.
- [11] Cole Stryker and Ivan Belcic. ¿qué es el ajuste fino eficiente de parámetros (peft)?
- [12] Dave Bergmann. ¿qué es el fine-tuning?
- [13] Shiqi Chen, Yiran Zhao, Jinghan Zhang, I-Chun Chern, Siyang Gao, Pengfei Liu, and Junxian He. Felm: Benchmarking factuality evaluation of large language models, 10 2023.